RankMixer: Scaling Up Ranking Models in Industrial Recommenders (2025)

arXiv:2507.15551v3 (cs.IR), 26 Jul 2025.

Challenges

- **CPU-era feature-crossing modules fail to exploit modern GPUs:** low MFU (4–5%) and poor parallelism hinder scaling in production ranking.
- **Strict latency and QPS constraints:** industrial recommender systems must serve high QPS under tight latency budgets, limiting naive parameter scaling.
- **Imbalanced expert training in Sparse-MoE:** dynamic routing and load imbalance can starve experts and degrade performance at scale.

Key Initiatives

- **Hardware-aware feature interaction (TokenMixer):** replace self-attention with a multi-head token mixing module for GPU-friendly parallel cross-feature fusion.
- **Per-token FFNs for subspace modeling:** allocate separate FFNs per token to capture distinct feature subspaces and avoid parameter sharing bottlenecks.
- **Sparse-MoE extension for 1B+ scale:** integrate dynamic routing (DTSI-ReLU) to scale to 1B parameters with minimal latency impact and balanced expert training.
- **Unified transformer-like blocks:** ranker architecture with residual connections + LayerNorm + TokenMixer + PFFN blocks achieves high MFU and compute efficiency.

Methodology

- **RankMixer block (Figure ??):** each block applies a Multi-Head Token Mixing layer followed by a Per-Token Feed-Forward Network, both wrapped in residual + LayerNorm.
- **Sparse-MoE variant:** activate a subset of per-token FFNs via dynamic routing, boosting capacity 70× with only a 20× FLOPs increase (MFU soared 4.5%→44.6%).
- **Scaling directions:** width (D), token count (T), and layers (L) can be scaled orthogonally following Transformer-like scaling laws.

Experiments

- **Offline performance/efficiency (Table 1):** RankMixer-100M outperforms SOTA cross-structure models (+0.64% AUC / +0.72% UAUC) with 233G FLOPs vs baselines at up to 442G.
- **Component ablations (Tables 2 & 3):** Multi-Head Token Mixing and per-token FFNs are critical (–0.50%/–0.31% AUC w/o);
- **Online A/B lift (Tables 4–6):** 1B model improved feed active days +0.29%, app duration +1.08%; advertising AUC +0.73%, revenue +3.90%. MFU ↑10×, latency flat.
- **Scalability & expert balance (Figures 2–4):** dynamic Sparse-MoE retains accuracy under 1/8 activation ratio; consistent scaling laws across Params/FLOPs.

References (selected)

- Transformer scaling laws: Hoffmann et al., 2022; Kaplan et al., 2020.
- Sparse Mixture-of-Experts routing: Lepikhin et al., 2020; Shazeer et al., 2017.
- Cross-feature interaction: Guo et al., 2017 (DeepFM); Tang et al., 2019 (DCNv2); Luan et al., 2023 (CrossFormer).
- Modern scaling in RecSys: Zhang et al., 2024 (Wukong); Zhai et al., 2025 (HSTU); Rajput et al., 2023 (TIGER).

cross-token feature interactions only through the parameter-free operator. This strategy outperforms the self-attention mechanism in terms of performance and computational efficiency. 2. *Per-token feed-forward networks (FFNs)* expand model capacity significantly and tackle inter-feature-space domination problem by allocating independent parameters for different feature subspace modeling. These FFNs also align well with the recommendation data patterns, enabling a better scaling behavior. To further boost the ROI of large-scale models, we extend the per-token FFN modules into a *Sparse Mixture-of-Experts (MoE)* structure. By dynamically activating only a subset of experts per token for different data, we can significantly increase the model capacity with a minimal increase in computational cost. RankMixer adopts a highly parallel architecture similar to transformers, but overcomes several critical limitations of self-attention based feature-interaction: low training efficiency, the combinatorial explosion when modeling cross-space ID similarity and severe memory-bound caused by attention weights matrix. At the same time, RankMixer offers greater model capacity and learning capability under the same Flops compared with Vanilla Transformer.

In production deployment on Douyin's recommendation system, we demonstrate that it is feasible to scale model parameters by over 100× while maintaining shorter inference latency compared with previous baseline. This is made possible by the RankMixer architecture's ability to decouple parameter growth from FLOPs, and decouple FLOPs growth from actual cost through high MFU and engineering optimization.

The main contributions can be summarized as follows:

- We propose a novel architecture called RankMixer follows a hardware-aware model-design philosophy. We design the multi-head token mixing and per-token FFN strategies to capture heterogeneous feature interactions efficiently, and using dynamic routing strategy to improve the scalability of SparseMoE in RankMixer.
- Leveraging levers of high MFU and performance-optimization, we scale the model parameters by 70× without increasing the inference cost, including improving MFU and Quantization.
- We conduct extensive offline and online experiments and investigate the model's scaling law on the trillion-scale industrial recommendation dataset. The RankMixer model has been successfully deployed on the Douyin Feed Recommendation Ranking for full-traffic serving, achieving 0.3% and 1.08% increase on active days and App duration, respectively.

2 Related Work

Modern recommendation systems are based on Deep Learning Recommendation Models (DLRMs) and how to effectively model the feature interactions is a crucial factor for DLRMs [5, 11, 15, 22, 31, 33, 39]. Wide&Deep [5] is one of the earliest efforts, which combines the logistic regression (wide part) and DNN (deep part) to capture low-order and high-order feature interactions respectively. DeepFM [11] is another achievement, which integrates the Factorization Machine (FM) and the DNN. In addition, DeepCross [26] is an extension of the residual network [13], aiming to learn automatic feature interactions implicitly. While it has proven to be quite challenging to simply rely on DNNs to learn high-order feature interactions [22, 25]. Explicit cross methods design different operators to explicitly capture high-order feature interactions, such as PNN [22], DCN [32] and its successor DCNv2 [33], xDeepFM [18], FGCNN [20] and FiGNN [17]. AutoInt [28] and Hiformer [10] employ attention mechanism with residual connections to learn complex interactions. DHEN [39] proposes to combine multiple interaction operators together. Despite improving accuracy, these new architectures increase the model latency and memory consumption and their model size is relatively small.

Scaling law has emerged as a fundamental theme in deep learning and a pivotal catalyst for numerous breakthroughs over the past decade, particularly in Natural Language Processing (NLP) [14, 16], Computer Vision (CV) [8, 37], and multi-modality modeling [23, 24], which describe the power-law correlations between model performance and scaling factors, such as model size, data volume, and computational capacity [1, 16, 29, 30]. Recently, scaling laws in recommendation systems have drawn much attention from researchers [12]. Studies have explored the scaling strategy for pre-training user activity sequences [6], general-purpose user representations [27, 40] and online retrieval [9, 34]. Wukong [38] stacks FMs and LCB to learn the feature interactions. Orthogonally, Zhang et al. [40] scaled up a sequential recommendation model to 0.8B parameters. HSTU [36] enhances the scaling effect of Generative Recommenders (GRs) which focus more on the sequence part.

3 Methodology

3.1 Overall Architecture

RankMixer's overall architecture consists of T input tokens processed by L successive RankMixer blocks, followed by an output pooling operator. Each RankMixer block has two main components: (1) a Multi-Head Token Mixing layer, and (2) a Per-Token Feed-Forward Network (PFFN) layer, as illustrated in Figure 1 . First, the input vector $\mathbf{e}_{\text{input}}$ is tokenized into T feature tokens $\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_T$, each representing a coherent feature vector. RankMixer blocks iteratively refine token representations for L layers through:

$$\begin{aligned} \mathbf{S}_{n-1} &= \text{LN}\left(\text{TokenMixing}\left(\mathbf{X}_{n-1}\right) + \mathbf{X}_{n-1}\right), \\ \mathbf{X}_n &= \text{LN}\left(\text{PFFN}\left(\mathbf{S}_{n-1}\right) + \mathbf{S}_{n-1}\right), \end{aligned} \tag{1}$$

where $\text{LN}(\cdot)$ is a layer normalization function, $\text{TokenMixing}(\cdot)$ and $\text{PFFN}(\cdot)$ are the multi-head Token Mixing module and the per-token FFN module, $\mathbf{X}_n \in \mathbb{R}^{T \times D}$ is the output of the n-th RankMixer block, $\mathbf{X}_0 \in \mathbb{R}^{T \times D}$ is stacked by $\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_T$, and D is the hidden dimension of the model. The output representation $\mathbf{o}_{\text{output}}$ derives from the mean pooling of final layers's representations $\mathbf{X}_L$ which will be used to compute the different task's predictions.

3.2 Input Layer and Feature Tokenization

The first thing to build a large-scale recommendation model is to prepare inputs with abundant information, such as **User features** include user ID and other user information, **candidate features** include video ID, author ID etc., **Sequence Features** processed through Sequence Module [4, 42] to capture temporal interests, yielding $\mathbf{e}_s$ and **Cross Features**. the cross features between the user and the candidate. All the features will be transformed into embeddings with diverse dimensions.

Figure 1: Architecture of a RankMixer block (Figure 1).